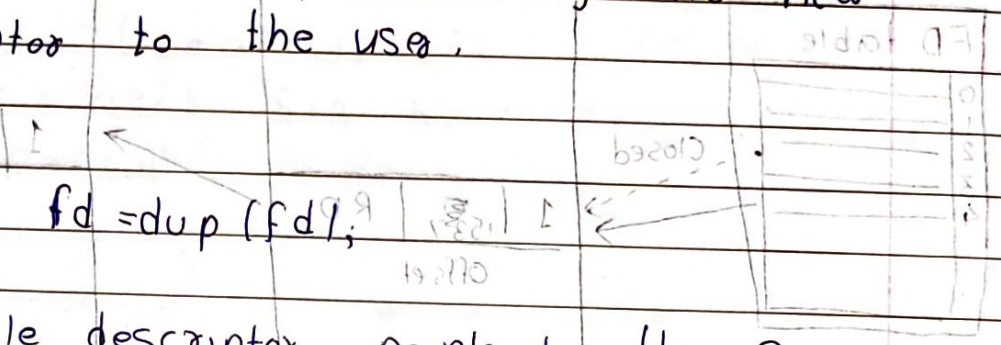
★ DUP

The dup System call copies a file descriptor into the first free slot of the user file descriptor table, returning the new file descriptor to the user.



Syntax:

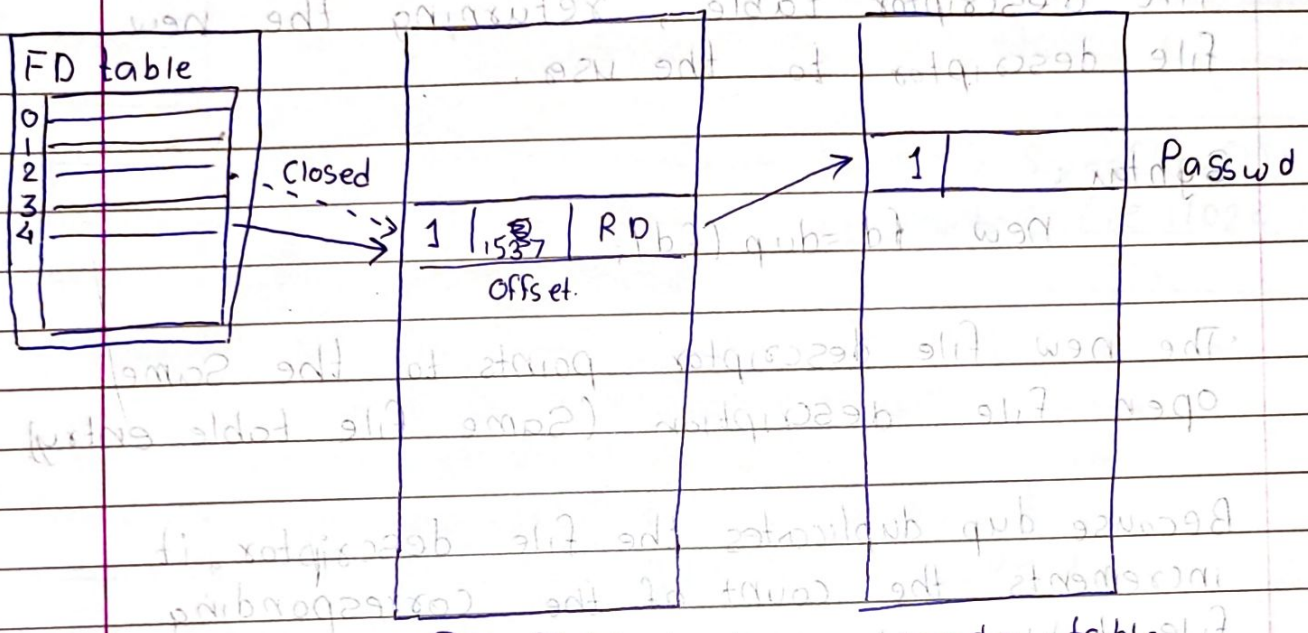
```
new fd = dup (fd);
```

- The new file descriptor points to the same open file description (same file table entry)

Because dup duplicates the file descriptor, it increments the count of the corresponding file table entry, which now has one more file descriptor entry that points to it

ex.

```
#include <fcntl.h>
int main()
{
    int i, j;
    char buf[512], buf2[512];
    i = open ("/etc/passwd", O_RDONLY);
    j = dup(i)
    read (i, buf1, sizeof (buf1));
    read (j, buf2, sizeof (buf2));
    close (i)
    read (j, buf 2, sizeof (buf2));
}
```

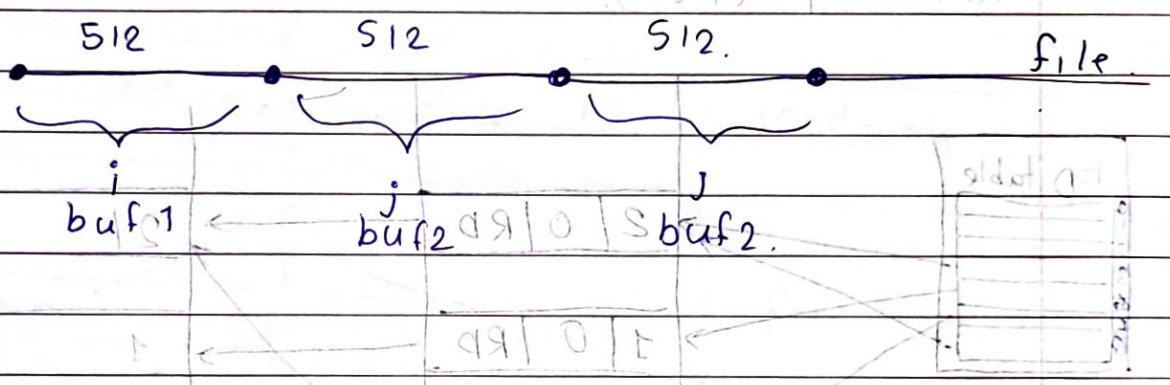


* First Read (read (i, buf1, sizeof (buf1))):
  - i and j share the same file offset (because dup() does not create a new file description, just a new descriptor pointing to the same underlying file).
  - i reads the first 512 bytes from /etc/passwd.
  - The offset moves forward by 512 bytes.

* Second Read (read (j, buf2, sizeof (buf2))):
  - Since j shares same file offset with i, it starts after the first 512 bytes.
  - So, buf2 will contain next 512 bytes from /etc/passwd, not same as buf2.

* On closing (i).
Closing i does not affect j, because they are just references to same file.

* Third read (read (j, buf2, sizeof (buf2))
• It reads next 512 bytes (ie 1024-1536) of /etc/passwd.



* Open * Difference between Open and dup()
• Open creates new entry in file table
• dup does not create new entry.
• In open reference count is is incremented
• In dup reference cound is as it is but FD count is increased.
• Open opens a new file and returns a file discriptor whereas duplicates an existing file descriptor. *

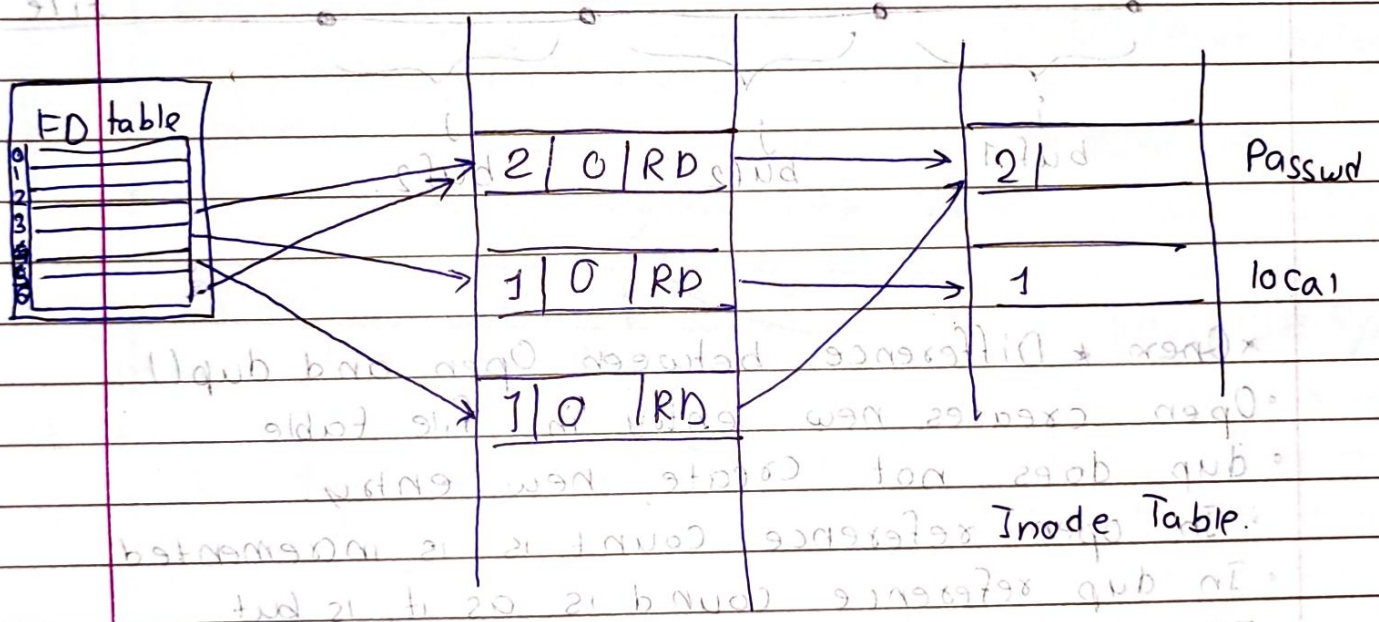
* dup2(): It is an enhanced version of dup() that allows you to duplicate a file descriptor to a specific number.

dup2 (4, 1).
↳ Close the 1st FD
↳ dup the FD to 4 pointing to file table entry

fd = open ("output.txt", O_WRONLY);

dup2 (fd, 1) redirects stdout (fd 1) to output.txt

fd1 = open ("../passwd", O_RDONLY);
fd2 = open ("../local", O_RDONLY);
fd3 = open ("../passwd", O_RDONLY);
fd4 = dup (fd1);



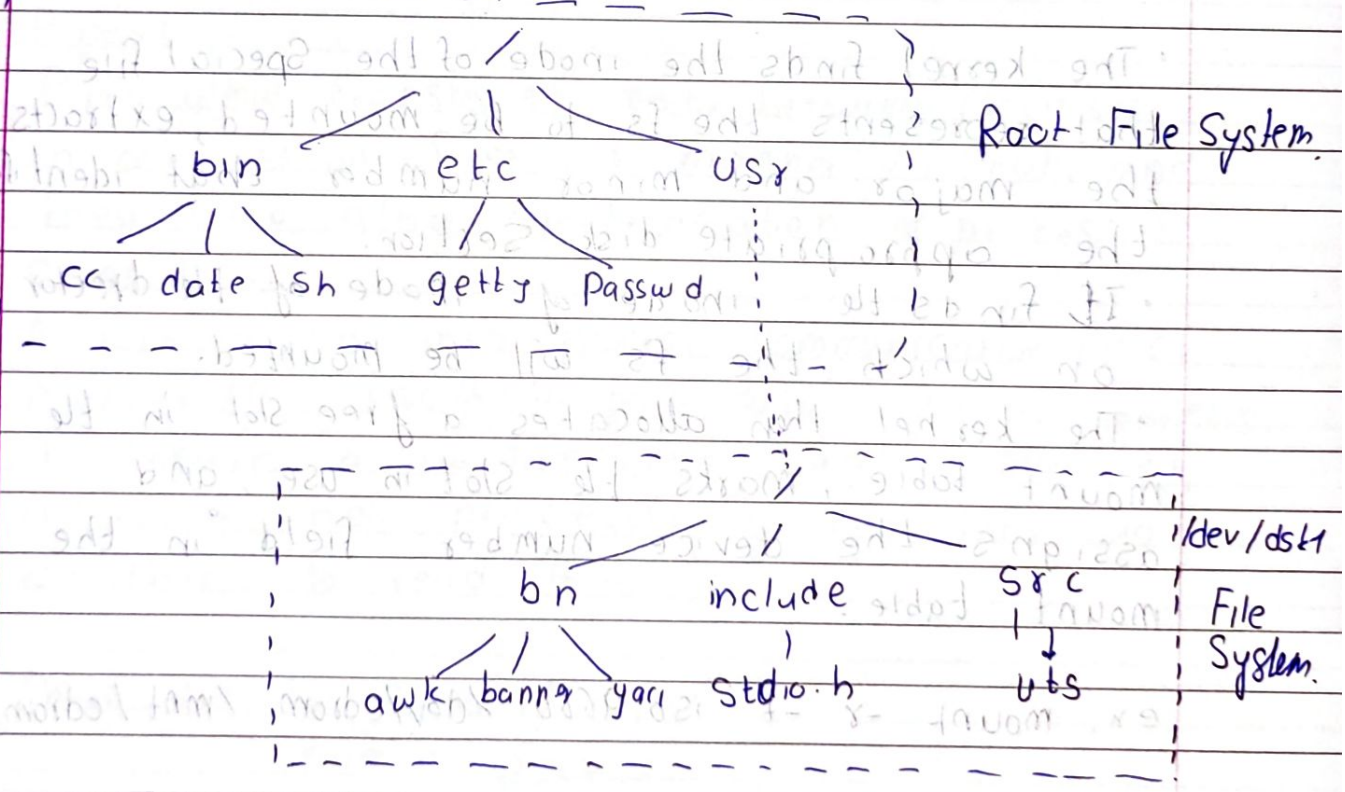
* Mounting and Unmounting File Systems.
- The mount system call connects the file system in a specified section of a disk. to the existing file system hierarchy.
- Unmount System call disconnects a file system from the hierarchy

- The mount system call thus allows user to access data in a disk section as a file system.

Syntax:

mount (Special pathname, directory pathname, options);

- where special pathname is the name of the device special file of the disk section containing the fs to be mounted.
- directory pathname is the directory in the existing hierarchy where the fs will be mounted (called mount point)
- Options indicate whether fs is read-only.



FS Tree before & After mount

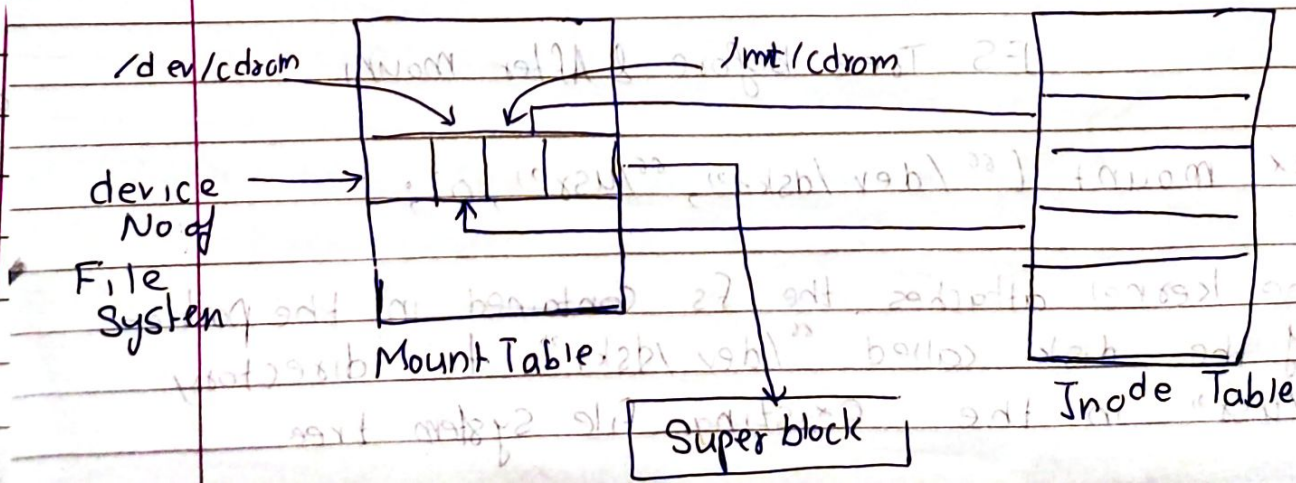
ex. mount ("/dev/dsk1", "/usr", 0);

the kernel attaches the fs contained in the portion of the disk called "/dev/dsk1" to directory "/usr" in the existing file system tree

The kernel has a mount table with entries for every mounted file system.
each mount table entry contains.
- a device no that identifies the mounted fs.
- a pointer to a buffer containing the fs super block.
- a pointer to the root inode of mounted fs
- a pointer to inode of directory that is the mount point.

- The kernel finds the inode of the special file that represents the fs to be mounted, extracts the major and minor number that identify the appropriate disk section.
- It finds the inode of inode of the director on which the fs will be mounted.
- The kernel then allocates a free slot in the mount table., marks the slot in use, and assigns the device number field in the mount table.

ex. mount -r -t iso9660 /dev/cdrom /mnt/cdrom.



-r → Mounts in read only mode
-t iso9660 → Specifies fs type ISO 9660 (used for CDs / DVDs)
/dev/cdrom → device file representing CD Rom.
/mnt/cdrom → mount point

Q Name of PCB in linux → task_struct
Q Name of file table → Open file table
Q Name of mount table → fs tab (/etc/fstab)

* Pipes:
- Pipes allow transfer of data between processes in a first-in-first out manner (FIFO), and they also allow synchronization of process execution.
- A pipe is an inter-process communication (IPC) mechanism used to pass data between processes.
- It creates a unidirectional data channel, allowing one processs to write data and another to read it.
- There are two kinds of pipes.
  - named pipes
  - unnamed pipes.
- Processes use open system call for named pipes, but the pipe system call to create an unnamed pipe.

- Afterwards, processes use the regular system calls for files, such as read, write, and close when manipulating pipes.
- Only related processes (descendants of a process that issued the pipe call) can share access to unnamed pipes.
- However, all processes can access a named pipe regardless of their relationship

* Pipe System Call

Syntax:

```
pipe(fdptr);
```

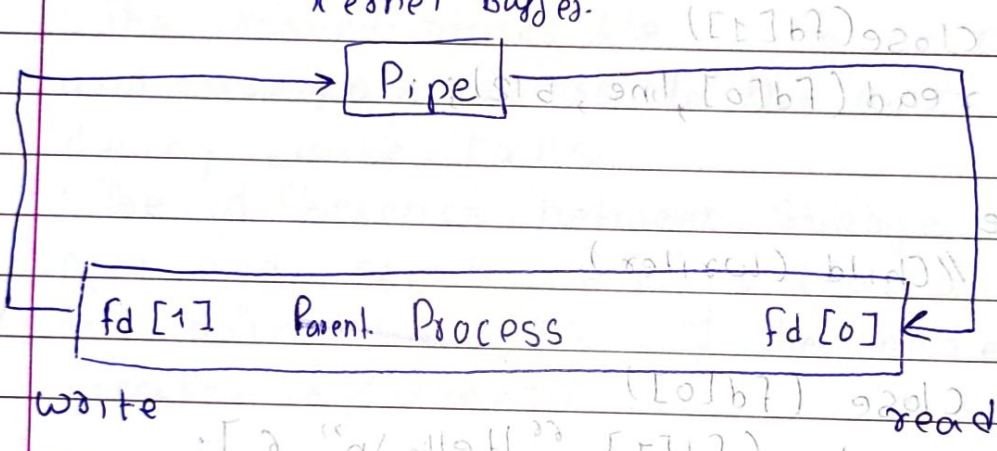
where fdptr is the pointer to an integer array that will contain the two file descriptors for reading and writing the pipe.

- The pipe() system call is used to create a unidirectional communication channel between two processes, allowing one process to write data and another to read it.

fd[0] — read end of the pipe
fd[1] — Write end of the pipe.

- The kernel assigns an inode for a pipe using ialloc.
- The kernel then allocates two file table entries for the read and write descriptors, respectively, and updates the bookkeeping information in incore inode

- Each file table entry records how many instances of the pipe are open for reading or writing,
  - * initially 1 for each file table entry.
- Inode reference count indicates how many times the pipe was "opened", initially two - one for each file table entry.

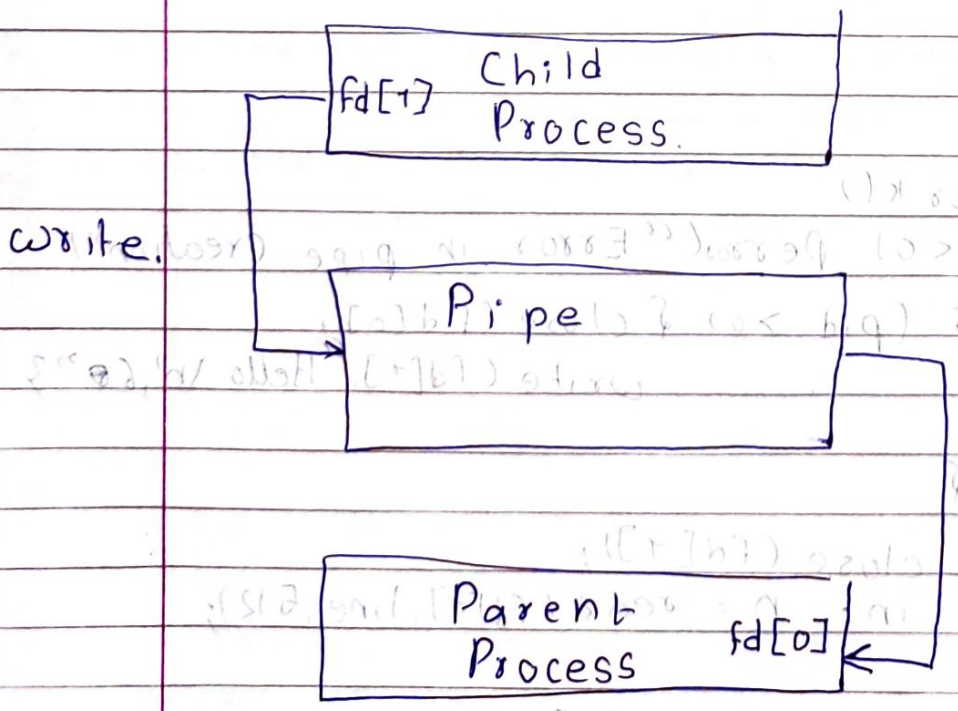


* Transfer data from parent to child.

```
int main(){
    int fd[2]
    pipe(fd);
    int pid = fork()
        if (pid < 0) perror("Error in pipe Creation");
        else if (pid > 0) { close(fd[0];
                            write(fd[1], "Hello \n", 60");
        else {
                close(fd[1]);
                int n = read(fd[0], line, 512);
    }
```

Transfer data from Child to Parent

```
int main(){
    int fd[2];
    char line[512];
    pipe(fd);
    int pid = fork();
    if (pid<0)  perror("Error in pipe creation")
    else if (pid >0)
      //Parent (read)
    {
       close(fd[1])
       read(fd[0], line, 512);
    }
    else
       //Child (writer)
    {
       close (fd[0])
       write (fd[1], "Hello \n", 6);
```



* Reading and Writing in Pipes
- A pipe should be viewed as if processes write into one end of the pipe and read from the other end.
- Process access data from a pipe in FIFO manner, meaning that the order that data is written into the pipe is the order in which it is read from the pipe.

- The kernel stores the data on pipe device and assigns blocks to the pipe as needed during write calls.
- The difference between Storage allocation for pipe and regular file is that a pipe uses only the direct blocks of the inode for greater efficiency

- When a process reads a pipe, it checks if the pipe is empty or not. If it contains data, the kernel reads data similar to regular files using algo read.

- If a process attempts to read more data than is in the pipe, the read will complete successfully after returning all data, even though it does not satisfy the user count.

- If a process writes a pipe and pipe cannot hold all the data, the kernel marks the inode and goes to sleep waiting for data to drain by some read(). During read kernel will notice proces that are asleep and waiting for data to drain and will awaken them

- When writer process has written all the data, the kernel updates the pipe's (inode) write pointer so that next process can write the pipe from where the current finished

* Closing Pipe
- When closing a pipe, a process follows the som precedure as for regular file, except that kernel does special processing before releasing the pipe's inode
- The kernel decrements the number of pipe writers or readers.
- The kernel frees all its data block if count of writer and reader is 0 and no process are asleep.
- The kernel then indicates pipe is empty.

- Close() function in pipe is used to close unused ends of the pipe, ensuring proper data flow and preventing deadlocks.

* popen() and pclose().
- The popen() function is used to execute a shell command and create a pipe to read its output or write input into it.
- The pclose() function closes the pipe and waits for the command to complete.

Page No.:
Date:

Syntax :

File * popen(const command, "r");
File * Popen(command, "w");

int plose (File *stream)

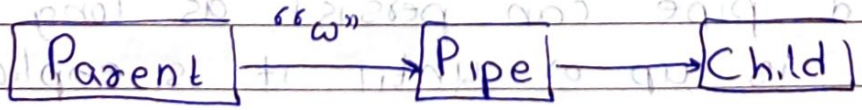
command : Shell commands to execute
modes - "r" read from command's output
- "w" write to command's input

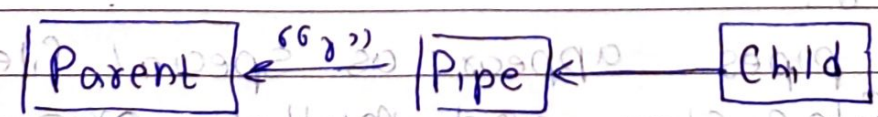
• Returns A File pointer to interact with process

Working of popen

1. Creates a pipe
2. Fork.
3. In Child
- Closes unnecessary file descriptors
- executes command using execl()

4. In parent
- returns File* stream that can be used with fgets(), fprintf()
- Close unused end of pipe.





```
int main()
{
    File * fp;    // file pointer
    fp=popen("/bin/ls", "r")  //Command to exec
    Char buffer[100];

    While (fgets(buffer, sizeof(buffer), fp)!=NULL)
        printf("%s", buffer);
    pclose(fp)
    return 0;
}
```

Output:
This code executes /bin/ls in child process and the output of this command is read through FILE and printed

* Named Pipes (FIFO)
- FIFO is a named pipe that allow IPC between unrelated processes.
- Unlike pipe(), which only between parent-Child processes.
- FIFO enables communication between separate processes.
- While a traditional pipe exists temporarily, a named pipe can persist as long as the system is up or until it is explicitly deleted.
- Named pipes appear as special files in the filesystem, and multiple processes can attach to them for reading and writing, facilitating inter-process communication.

- This file type is created using the mkfifo() system call in C.
- Once created, any process can open the named pipe for reading or writing, similar to ordinary file.

```
int mkfifo (pathname, mode);
```

Both mkfifo and mknod can be used to create a FIFO (Named Pipe)

1. mkfifo: It is specially designed to create named pipes

```
mkfifo /mydir/myfifo.
```

2. mknod: It can create device file, FIFOs and other special files.

```
mknod /mydir/myfifo p
```

The P argument tells mknode to create a FIFO

```
echo "Hello" > /mydir/myfifo //Writes
cat /mydir/myfifo //Reads.
```

* Tee Command:

The tee command reads from Standard input (Stdin)

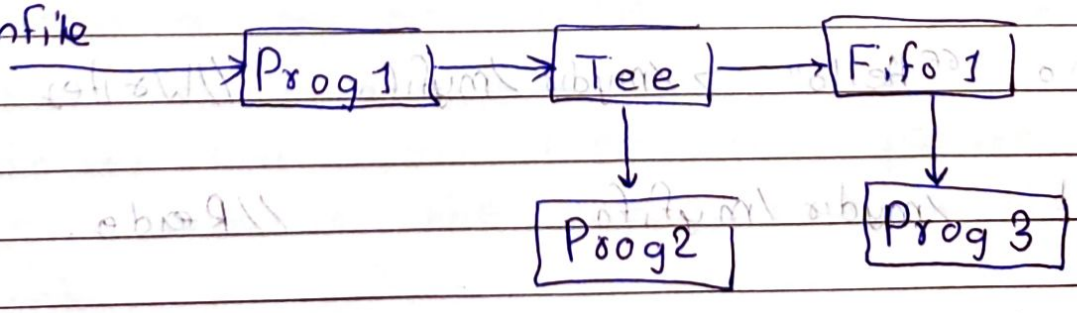
and writes to both standard output (Stdout) and a file (or multiple files) at same time.

ex. ls -l | tee a.txt
displays output of "ls -l" on terminal and saves to output.txt

mkfifo fifo1
prog 3 < fifo 1 &
prog 1 < infile | tee fifo1 | prog 2

* mkfifo fifo1 creates a named pipe fifo 1
* prog 3 < fifo 1 &
  - Runs prog3, taking input from fifo 1
  - & at end runs it in background so that next commands can proceed.

* prog 1 < infile | tee fifo1 | prog 2.
  - prog1 < infile : Runs prog 1 with input from infile.
  - | tee fifo1 : Sends output of prog 1 to both fifo 1 and prog 2.



mknod [options] name type major minor.

ex

mknod -m 620 /dev/p0 c 6 0

name → Path where device file is created

type → device type.

- c → Char device
- b → block device
- p → Named pipe.

major → Major number (identifies the driver).

minor → Minor number (specific instance of the device)

-m 620 → 6 → Owner → read & write
→ Group → write only.
→ 0 → others no permissions.

mknode -m 620 /dev/p0 c 6 0

mknod -m 620 /dev/p1 c 6 1

=

Other device.

- Both /dev/p1 and /dev/p2 will be handled by the same driver (Second instance of device)
- The driver will treat them as separate instances because the minor number is different.

* ls -l command
It lists detailed information about device files in /dev/

/dev/mem - Contents of memory.
/dev/kmem Contents of kernel's virtual memory
/dev/audio
/tty - keyboard input (Controlling terminal)
/tty0 - Main virtual consol. (first terminal)
/CPU. - CPU core info (/dev/CPU/0/*)
/net - network related virtual devices.
/ram0 - first ram disk
/dsp - direct Sound processing deve file for audio output

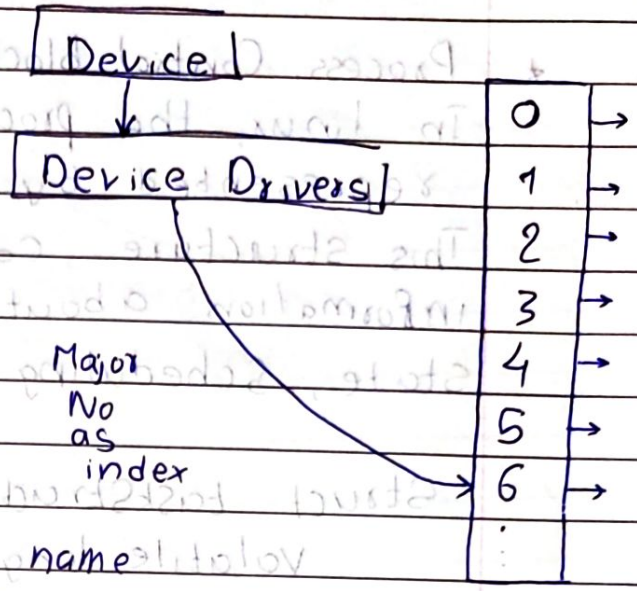
Q. When are device files created:
- At boot time. (detects hardwares.
- When a device is plugged in. (ex USb)
- Manually using mknode (Custom devices.
- Loading a kernel module.
- While mount for Pseudo-file System.

Q What are device files used for.
- It acts as interface to hardware and virtual devices
- They allow user-Space program to communicate with hardware devices and Software devices.

Q What are major & minor number
- It is used to identify device files in /dev/
Major no identifies driver that handles the device
Minor no identifies instance of device handle by

Date: YOUVA

Major No is an index of some array which contains pointers which points to certain functions



* Hard Link

A hard link is an additional name for a same inode (file data). It directly points to the same file on the disk.

* Symbolic (Soft) Link

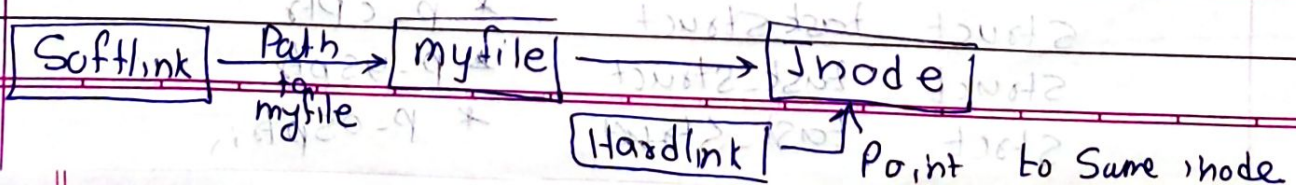
A Symbolic link is a special file that acts as a shortcut pointing to original file's path.

	Hard Link	Soft link
Points to	Inode	File path
Inode No	Same as original	different from the original
works after deleting Original	Yes	No.

* ln ~/myfile /etc/Somedir/sm

This command attempts to create a hard link from ~/myfile to /etc/Somedir/sm.

ln -s (Symbolic link).



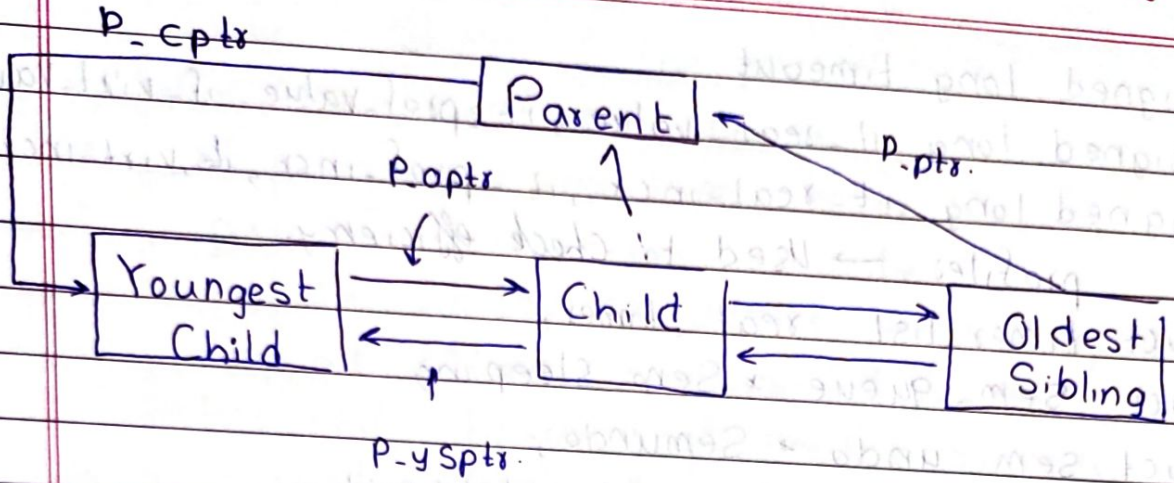
* Process Control block

In linux, the process control block (pcb) is represented by the task_struct structure. This structure contains all the necessary information about a process, including its state, scheduling details, memory information etc.

```
Struct taskStruct{
    volatile long State;
    volatile long Counter.
    Unsigned long rt_priority //read time
    unsigned long priority
    unsigned long policy
    unsigned long signal
    unsigned long blocked;
    Struct long blocked
    Struct sigaction sigaction[32];

Unsigned Flags : PF_ALIGNWARN
                 PF_TRACED
                 PF_TRACESYS
                 PF_STARTING
                 PF_EXITING.

int debug reg[8]
    Struct exec_domain * exec_domain
    Struct task_struct * next_task
    Struct task_struct * prev_task
    struct task struct * p_opptr,
    Strust task_Struct * p_pptr;
    Struct task_Struct * p_cptr
    Struct task_struct * p_ysptr
    Stact task_Struct * p_osptr;
```



Struct mm_struct mm;
unsigned long kernel_stack_page.
unsigned long saved_kernel_stack;
pid_t pid, pgrp, session, tgid_leader
unsigned short uid, euid, suid, fsuid;
unsigned short gid, egid, sgid, fsgid;
int groupset [NGROUPS]
struct fs_struct fs. → Structure related to file system
int count
unsigned struct unmask
Struct inode * root
Struct inode * pwd.

Struct files_struct files
long utime, stime, cutime, cstime, start_time;
↑ Process
↑ user mode
↑ kernel mode.

unsigned long timeout
unsigned long it_real_value, it_prof_value, it_virt_value;
unsigned long it_real_incr, it_prof_incr, it_virt_incr;
profiler → Used to check efficiency.
struct timer_list real_timer;
Struct sem_queue * Sem_sleeping
struct sem_undo * Semundo;
struct wait_queue * wait_childexit
Struct rlimit rlim [RLIM_NLIMITS],

int errno
int exit_code exit Signal
char comm [11] //Stores names of process.
unsigned long personality ; PER_LINUX
int dumpable → Stored in core file.
int did exec
struct desc_Struct * idt //Local descriptor table } for indices.
Struct linux_binfmt * binfmt //Use by loader
Struct thread_struct tss

#if def ... SMP ...
int processor, last_processor
int lock_depth
#t end if

- State - State of the process
- Counter - Referent Count
- rt_priority - real time priority
- priority : General priority
- policy. (FIFO/RR/OTHER/BATCH) represents policy
- Signal : Signal number of process
- blocked - bitmask of blocked Signals.
- Sigaction[32] - Array of 32 Sigaction, each represent how the process handles a Specific signal

- PF_ALIGNWARN : Used to warn about unaligned memory access
- PF_TRACED : Indicates that the process is being traced
- PF_TRACESYS : Use to track system call made by a process
- PF_STARTING : Indicates process is in initialization phase
- PF_EXITING : Indicates that the process is exiting.

(These Unsigned Flags define State or behavior of process)

debugreg[8] : An array of 8 debug registers used for hardware-assisted debugging

- exec_domain : Used for binary compatibility.
- next_task : Points to next process
- prev_task → Points to the previous process in global task.
- p_opptr (old parent ptr) → The original parent process before being reparented.
- p_pptr (Parent ptr) → Points to current parent
- p_cptr (child ptr) → Points to youngest child
- p_yspt (Youngest Sibling ptr)
- p_osptr (older Sibling ptr).

- mm_struct mm - Points to memory descriptor (Memory Mangement)
- kernel_stack_page, saved_kernel_stack;
  These hold process kernel stack
  - User-Space Stack
  - Kernel-Space stack.
  Saved kernel stack helps in context switch
- pid_t pid - Process id.
- pgrp - Process Group id.
- Session - Session id, groups all processes in a Session
- tgid - Thread Group id. (Shared among all threads of a process)
- leader → Indicates if this process is the leader of Session.
- uid - Real user id
- euid - Effective user id
- suid - Saved set user id
- fsuid - File System User id.
- gid - ~~real~~ group id
- egid - effective group id
- Sgid - Saved group id
- FSgid - file system Group id.
- groupset[NGROUPS] - Array storing additional group memberships

- Struct fs_struct *fs - Contains file System related info.
- Struct files_struct *files ; Stores file descriptors (fd table)

Date: / /

* Process Execution time
utime - user mode CPU time (User Space)
stime - System mode CPU time (Kernel Space)
cutime - Cumulative user mode time of all child process
cstime - Cumulative System-mode time of all child process
start_time - The time when process started.

* Process Timers (it fields)

timeout - defines when process should be scheduled next
it_real_value - Initial value of real-time timer.
it_prof_value - Time left for the profiling timer
it_virt_value - Time left for virtual timer
it_real_incr - Incr for real time timer
it_prof_incr - Incr for profiling timer
it_virt_incr - Incr for virtual timer

- profiler - Used for CPU profiling
  timer_list real_timer - The real-time timer

* Process Synchronization
  - sem_queue *sem_sleeping - Points to Semaphore queue
  - sem_undo - Used for undoing Semaphore operation
  - wait_child_exit - Queue where a parent process wait for child to terminate

* Resource Limits. (rlim[RLIM_NLIMITS])
  - Stores resource limits

* Process exit Information
  - err - Stores last error code.
  - exit_code - The process exit status.
  - exit_signal - The signal sent to the parent

* Process Identification
  - comm [11] - Stores names of process
  - personality - Defines process behavior
    eg: PER_LINUX, PER_SVR4

* Process Dumping.
  - dumpable - Control whether process memory can be dumped

* Execution Format
  - did_exec - Indicates if proc executed new binary
  - struct linux_binfmt * binfmt - Points to bin format handler
    eg ELF
    (used by loader.

* Processor - Specific Fields.
  - desc_struct idt - Interrupt Descriptor table.
    thread_struct tss - task State Segment.

* SMP (Symmetric Multiprocessing.
  - int processor
    last_processor - last CPU core used by process
    lock_depth - depth of kernel locking.
    (Spinlocks)

ELF- Executable & Linkable Format

↳ In linux / Unix.

created by compiler, assembers and linkers.

Q What is file format of MySQL & where is it stored?

/var/lib/mysql/

InnoDB engine .ibd

MyISAM engine - .MYD

Q Which is first process & created by whom.

init() - by kernel (after BRUB) start_kernel() function.

Q ls -l /dev/mem

displays details about physical memory access device in linux.

-l long listing format (detailed info).